

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

M Shakthi Sridevi
24011101063
AI DS A2

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224 .
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

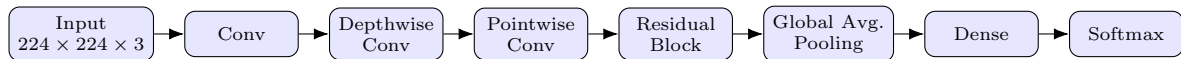
4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

GitHub Repository – Lab 5

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

The following initialization methods were compared:

1. Zero initialization.
2. Random initialization.
3. Xavier/Glorot initialization.
4. He/Kaiming initialization.

For this experiment, MobileNetV2 was created without pretrained weights. The final classifier was modified to output 37 classes.

The model was trained using Adam with a learning rate of 0.001.

Results

The results obtained from the notebook were:

Initialization	Training Loss	Validation Accuracy (%)
Zero	3.5882	0.00
Random	2.0341	0.00
Xavier	2.1055	0.00
He	–	–

The He initialization result should be entered after the corresponding experiment completes.

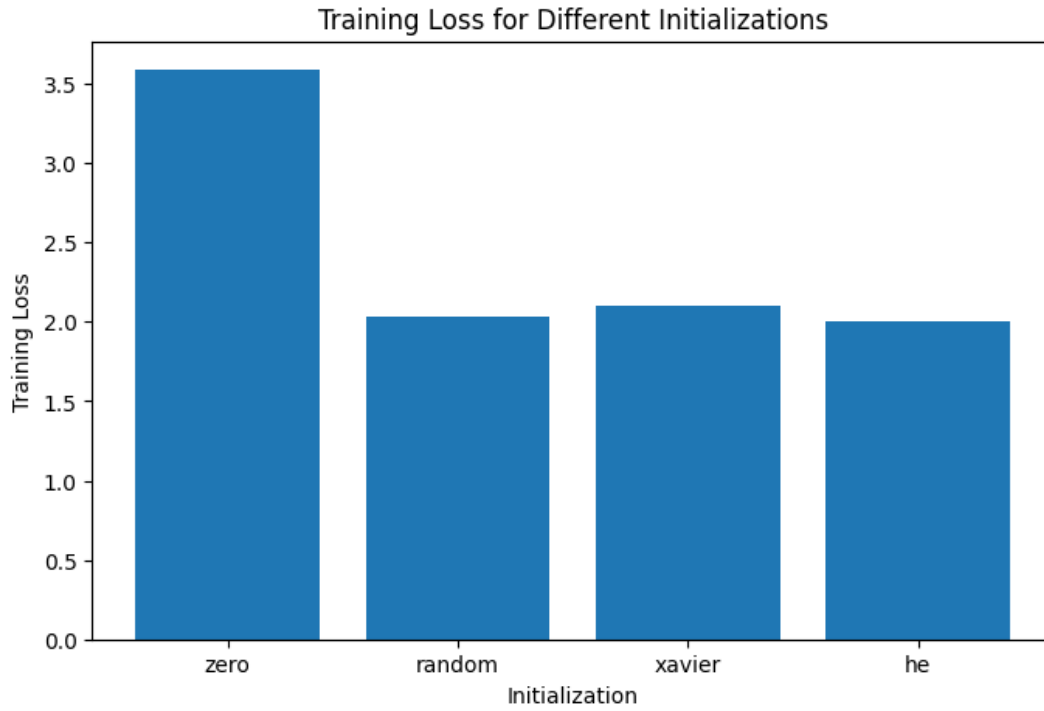


Figure 1: Training loss for different weight initialization methods.

Inference:

The experiment shows that initialization affects the starting point and optimization behaviour of a neural network. Since only one epoch and a small dataset subset were used, the results mainly demonstrate the initial learning behaviour rather than final model performance.

6. Regularization and Overfitting

The following regularization settings were compared:

- no regularization;
- L2 regularization;
- Dropout.

For L2 regularization, a weight decay of

$$10^{-4}$$

was used.

For Dropout, a dropout probability of

$$0.5$$

was used.

The pretrained MobileNetV2 feature extractor was frozen and only the classifier was trained.

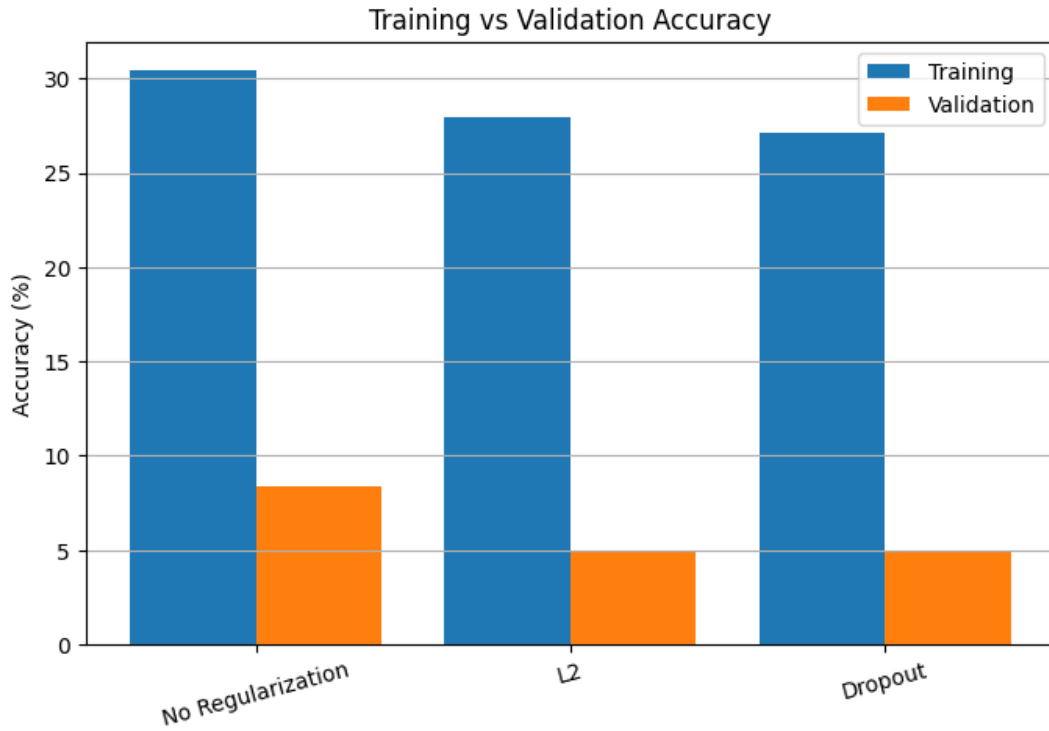


Figure 2: Training and validation accuracy for different regularization methods.

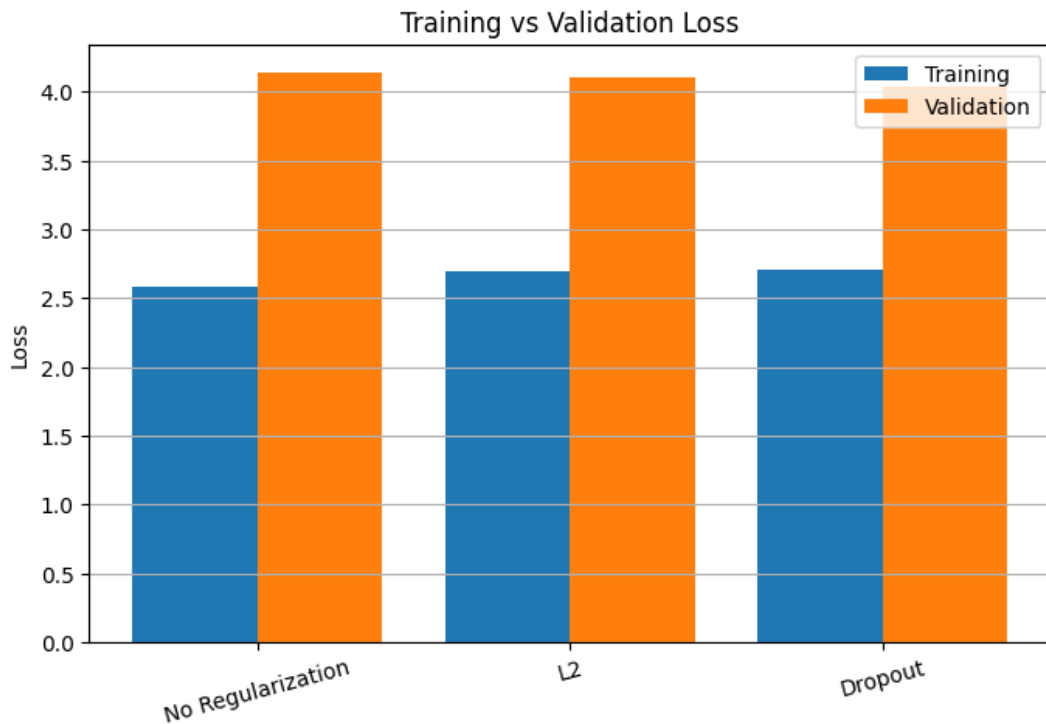


Figure 3: Training and validation loss for different regularization methods.

Inference:

L2 regularization penalizes large weights, while Dropout randomly removes activations during training. Both methods can reduce over-dependence on particular features and improve generalization.

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.

For a mini-batch

$$x_1, x_2, \dots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i.$$

The batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2 - 5)^2 + (4 - 5)^2 + (6 - 5)^2 + (8 - 5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ ,

$$\hat{x}_1 = \frac{2 - 5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4 - 5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6 - 5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8 - 5}{2.236} \approx 1.342.$$

Hence,

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Implementation

The notebook compares MobileNetV2 with Batch Normalization against a version where the Batch Normalization layers are replaced by identity layers.

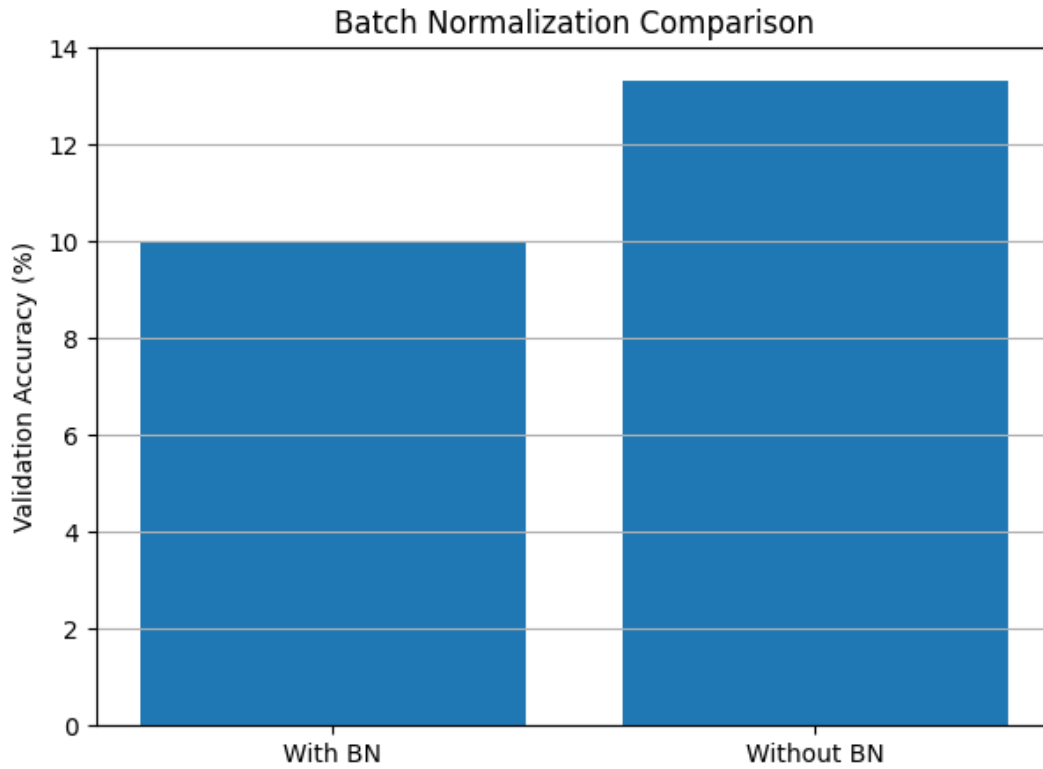


Figure 4: Validation accuracy with and without Batch Normalization.

Inference:

Batch Normalization produces approximately zero-centred and unit-scaled activations. The learnable parameters γ and β allow the network to learn an appropriate scale and shift.

8. Optimization Algorithms

The following optimizers were compared:

- SGD;
- Momentum;
- RMSProp;
- Adam.

All optimizers used a learning rate of 0.001.

The pretrained MobileNetV2 feature extractor was frozen and only the classifier was trained.

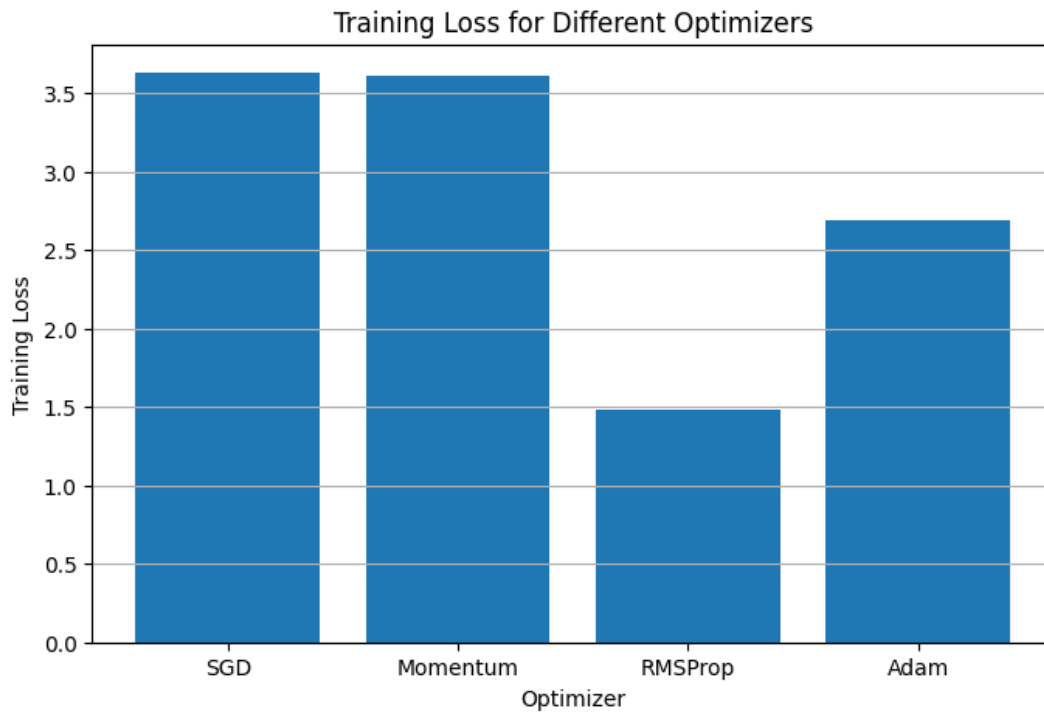


Figure 5: Training loss for different optimization algorithms.

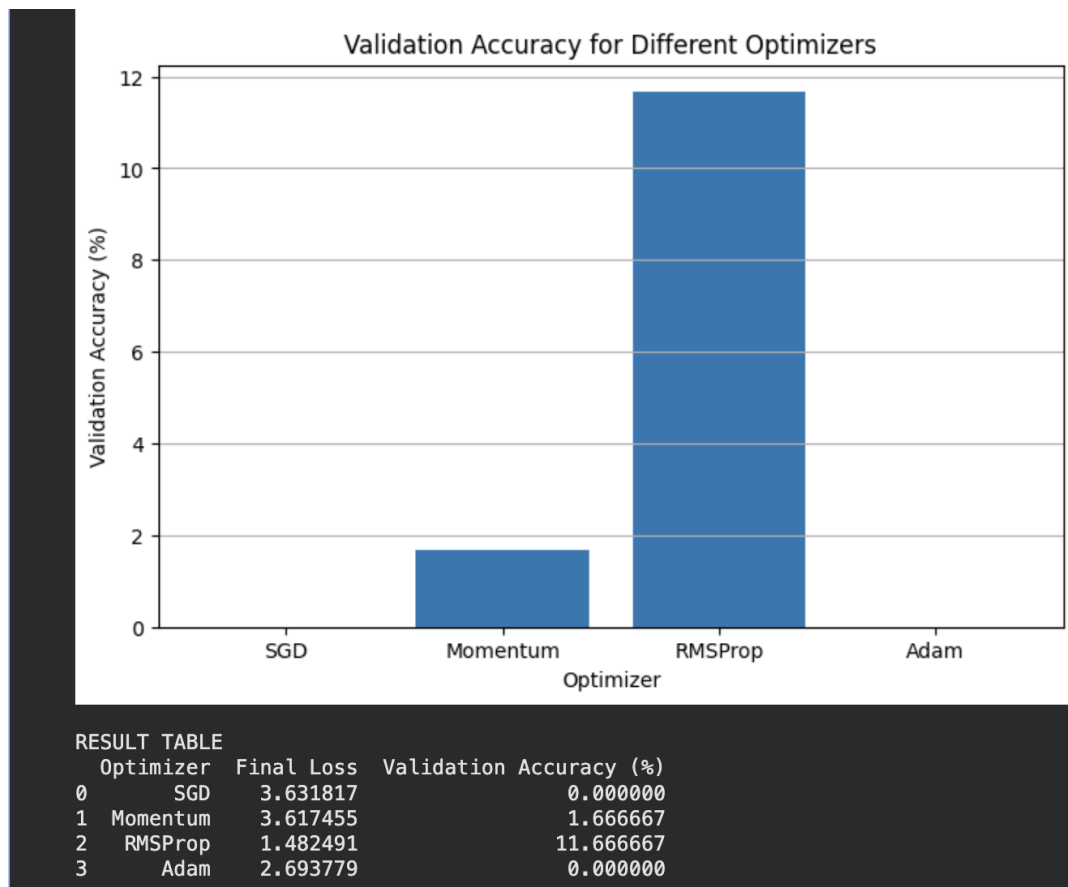


Figure 6: Validation accuracy for different optimization algorithms.

Inference:

Different optimizers use different parameter update strategies. Momentum can reduce oscillations in SGD, RMSProp adapts updates based on recent gradients, and Adam combines adaptive learning rates with momentum-like updates.

9. CNN Hyperparameter Tuning

The following hyperparameters were investigated:

Hyperparameter	Values Tested
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5

One hyperparameter was changed at a time while keeping the remaining settings fixed.

Learning Rate

The learning rate controls the size of parameter updates. A learning rate that is too large may cause unstable training, while a very small learning rate can result in slow convergence.

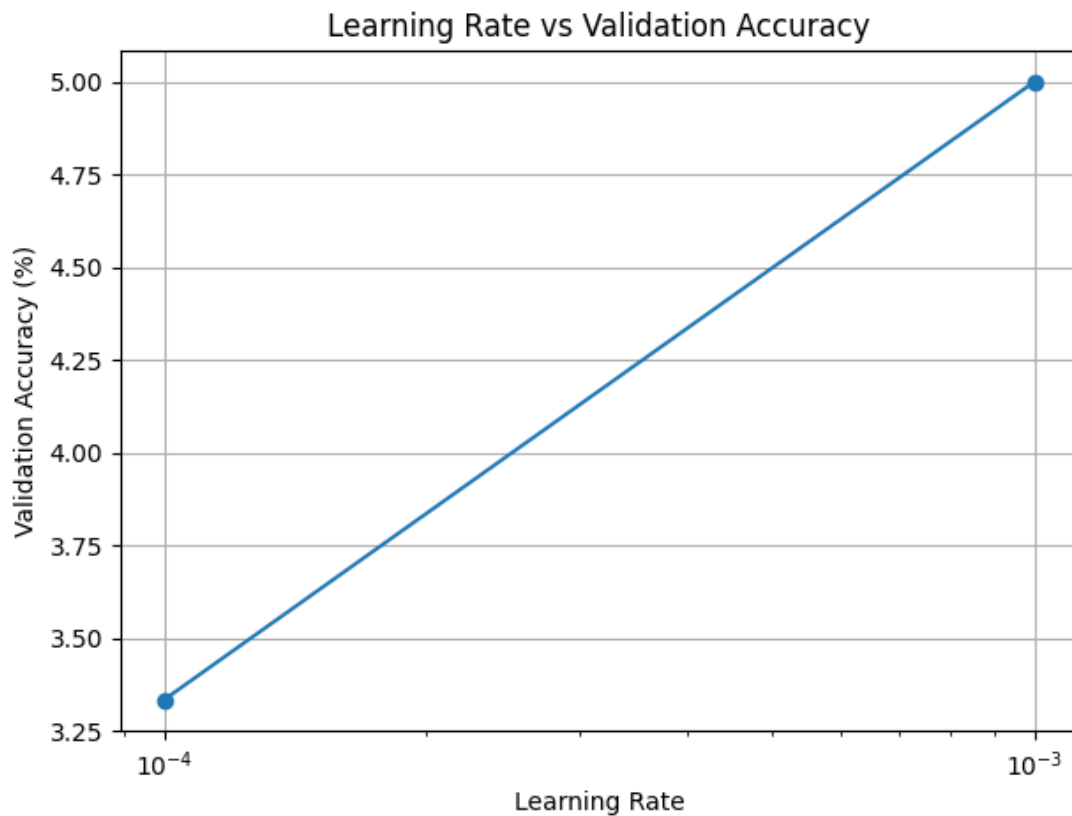


Figure 7: Effect of learning rate on validation accuracy.

Batch Size

Batch size determines the number of samples used to calculate each gradient update. Larger batches generally provide more stable gradient estimates but produce fewer updates per epoch.

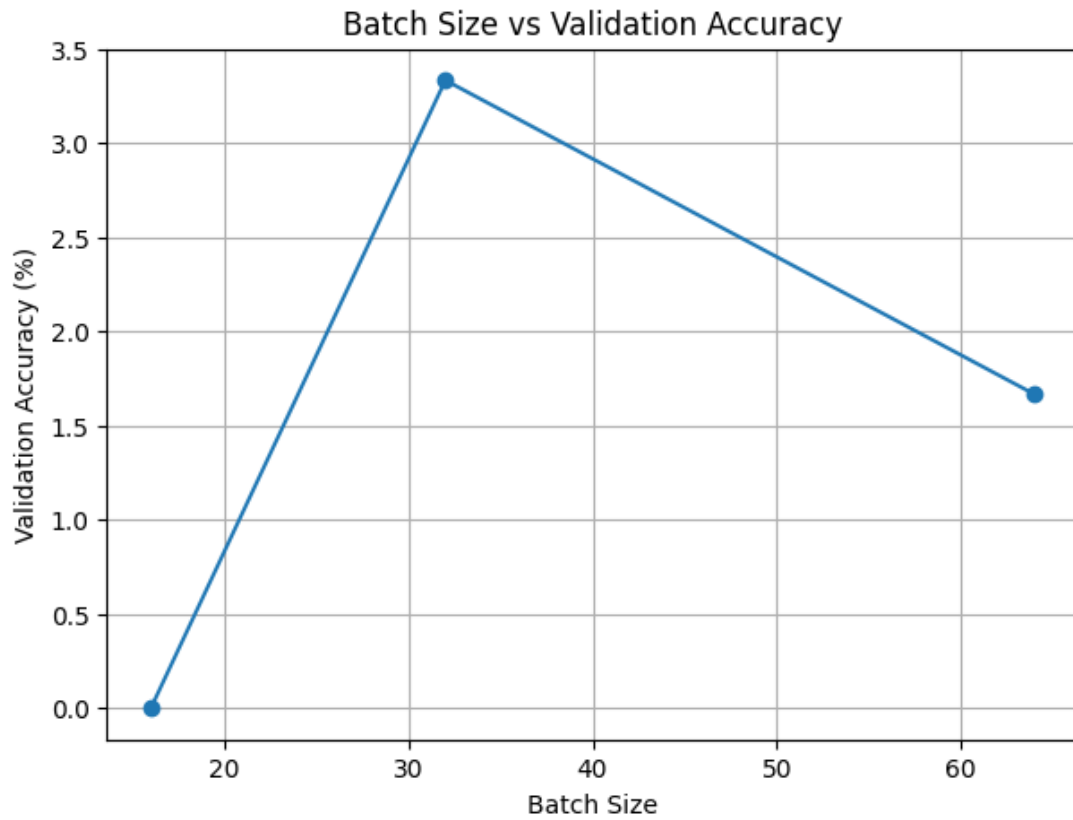


Figure 8: Effect of batch size on validation accuracy.

Dropout Rate

Dropout randomly disables a proportion of activations during training. It can reduce overfitting, although excessive dropout may remove too much useful information.

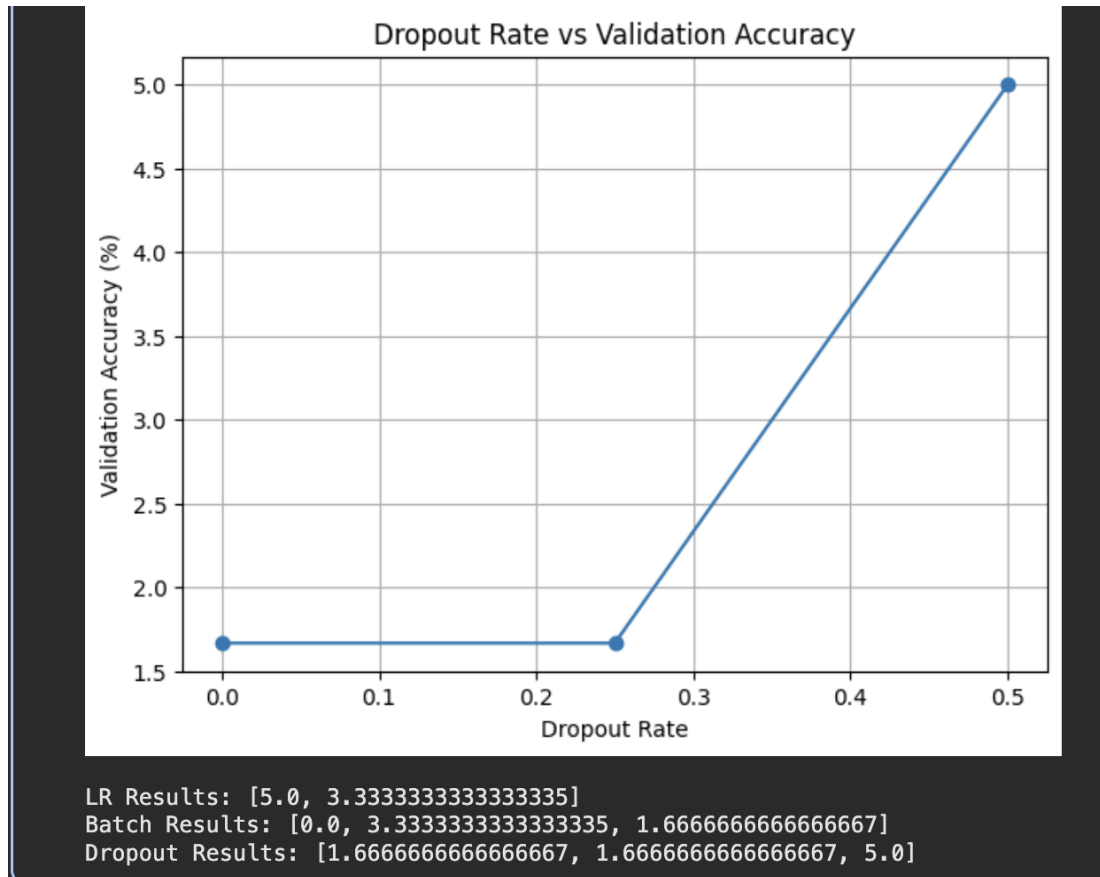


Figure 9: Effect of dropout rate on validation accuracy.

10. Transfer Learning and Fine-Tuning

MMobileNetV2 pretrained on ImageNet was used for transfer learning.

Case A: Feature Extraction

The feature extraction procedure was:

Pretrained MobileNetV2 → Freeze Base → New Classifier.

The pretrained convolutional layers were frozen and only the newly added classifier was trained.

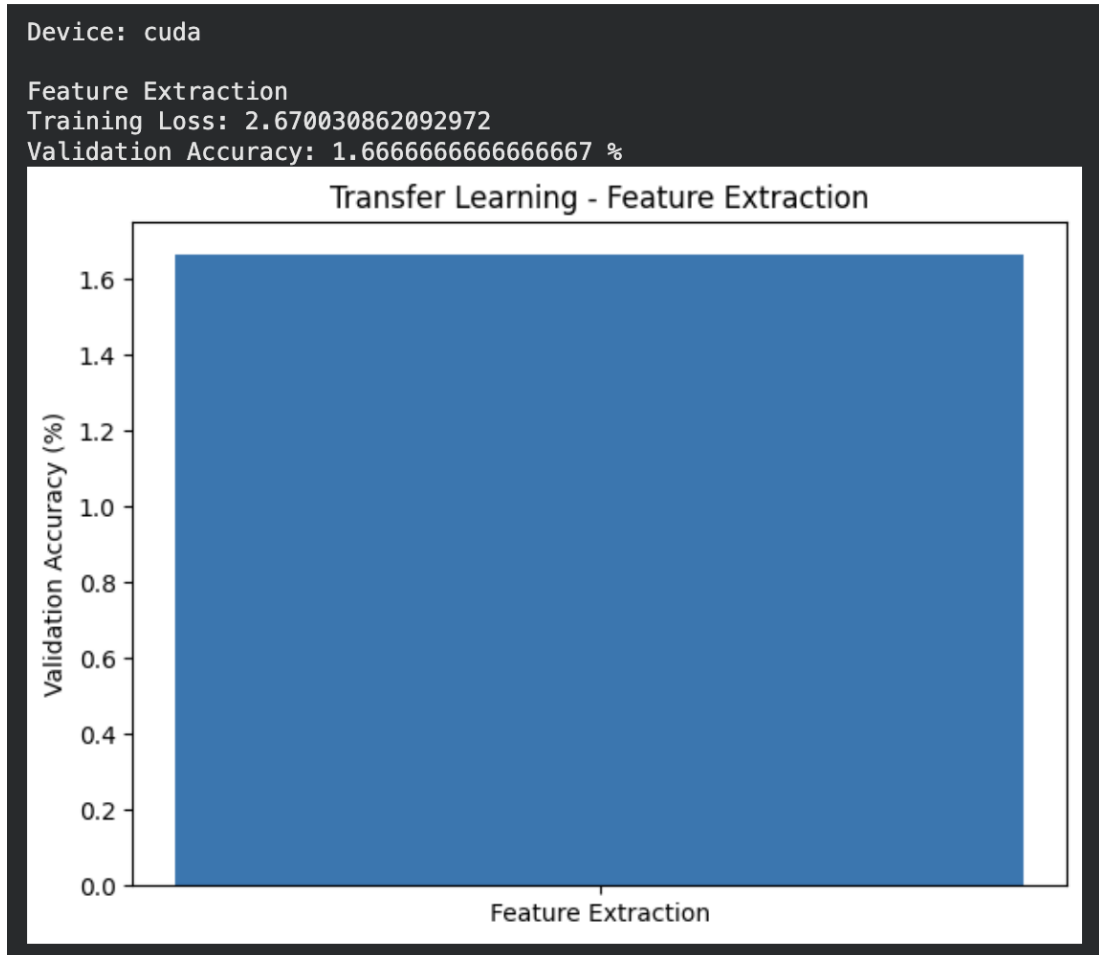


Figure 10: Validation accuracy obtained using feature extraction with pretrained MobileNetV2.

Case B: Fine-Tuning

For fine-tuning, the last two feature blocks of MobileNetV2 were unfrozen.

A smaller learning rate of

$$10^{-5}$$

was used.

The fine-tuning procedure was:

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate.

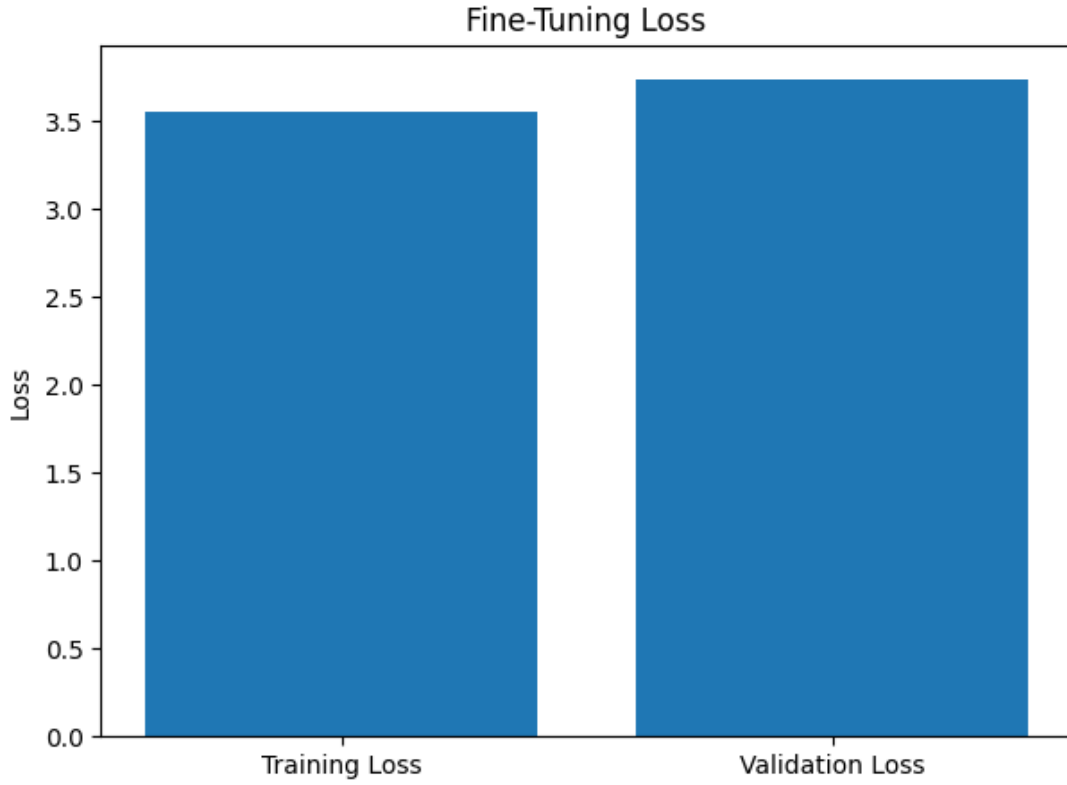


Figure 11: Training and validation loss during fine-tuning.

Inference:

Feature extraction trains only the newly added classifier, while fine-tuning also adapts selected pretrained layers. A smaller learning rate is used during fine-tuning to avoid changing useful pretrained features too aggressively.

11. K-Fold Cross-Validation

After the individual experiments, four promising configurations were compared using 5-fold cross-validation. The configurations used were:

Configuration	Learning Rate	Dropout
C1	10^{-4}	0
C2	10^{-4}	0.25
C3	10^{-5}	0.25
C4	10^{-5}	0.5

For this experiment, 200 images were used and divided into five folds.

For $K = 5$, the mean accuracy is

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i.$$

The standard deviation is

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

The result is reported as

Configuration	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Mean	SD
C1	5.0	5.0	10.0	10.0	5.0	7.0	2.45
C2	5.0	30.0	15.0	5.0	10.0	13.0	9.27
C3	2.5	5.0	0.0	0.0	7.5	3.0	2.92
C4	0.0	2.5	2.5	0.0	2.5	1.5	1.22

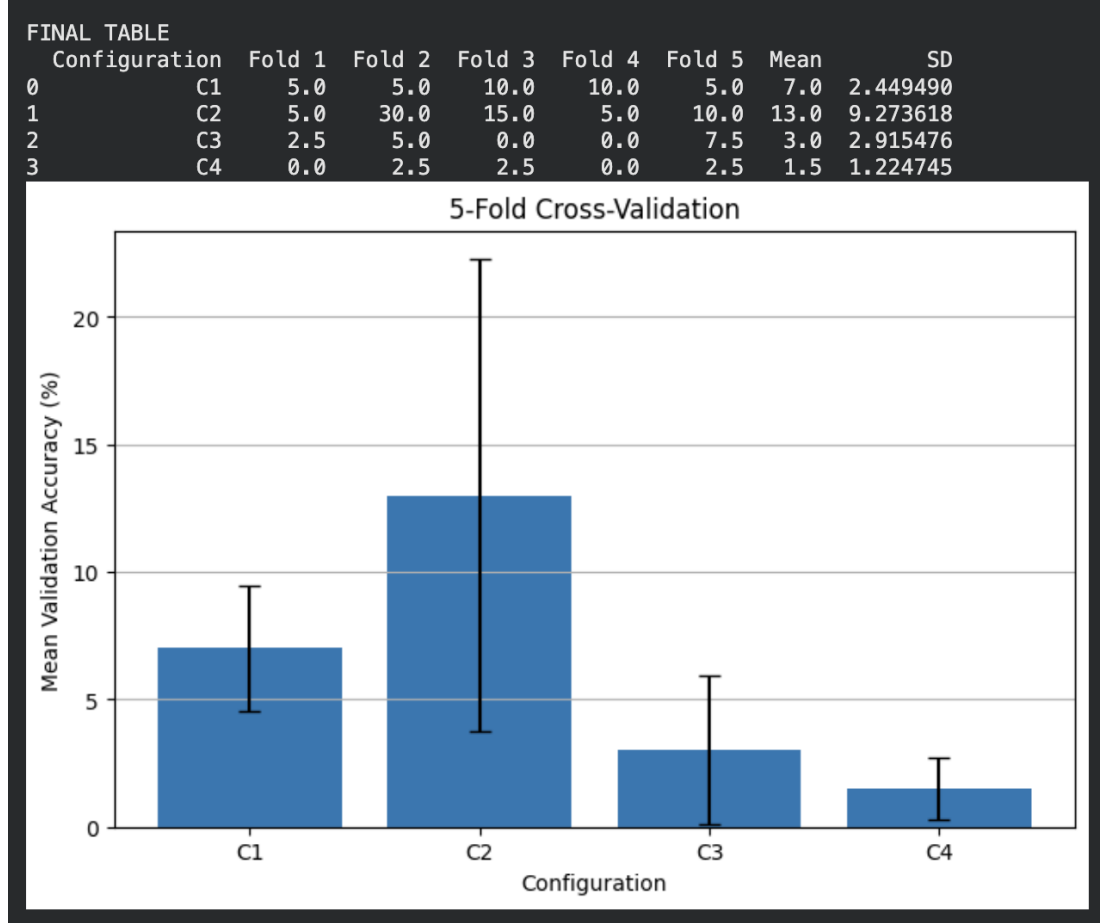


Figure 12: Mean 5-fold cross-validation accuracy with standard deviation error bars.

Inference:

5-fold cross-validation provides multiple validation measurements for each configuration. Therefore, both mean accuracy and standard deviation should be considered when selecting the best configuration.

12. Final Model Evaluation

After selecting the best configuration using cross-validation, the final model was evaluated on the independent test set.

The configuration used in the final notebook implementation was:

Learning Rate = 10^{-4} ,

Dropout = 0.25,

Batch Size = 32.

This corresponds to configuration C2.

The final model uses ImageNet-pretrained MobileNetV2 with the feature extractor frozen and a new classifier for 37 classes.

The following metrics were calculated:

- Accuracy
- Precision
- Recall
- F1-score
- Training Time
- Total Number of Parameters
- Trainable Number of Parameters

Metric	Value
Mean CV Accuracy	13.00%
CV Standard Deviation	9.27%
Test Accuracy	4.36%
Precision	0.0220
Recall	0.0435
F1-score	0.0182
Training Time	5.32 seconds
Total Parameters	2,271,269
Trainable Parameters	47,397

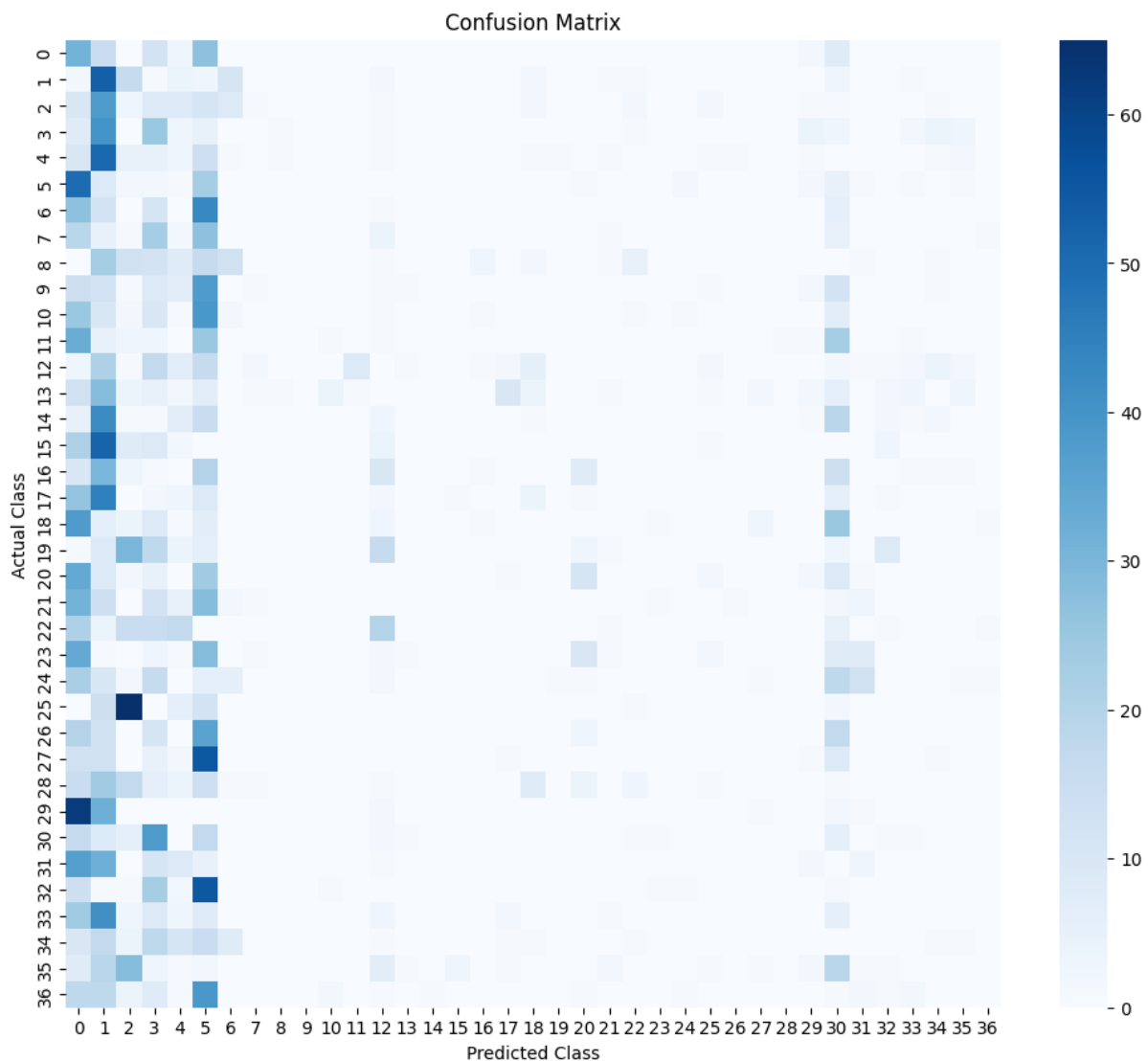


Figure 13: Confusion matrix for the final model on the test set.

Inference:

The confusion matrix shows which pet-breed classes are classified correctly and which classes are frequently confused.

Similar-looking breeds may be difficult to distinguish because of similarities in appearance, fur colour, body shape and facial features.

13. Overall Results

The overall experimental results are summarized below. The values are taken directly from the experiments performed in the notebook.

Configuration	CV Accuracy (%)	SD	Test Accuracy (%)	Training Time (s)
Baseline	7.00	2.45	4.36	5.32
Best Initialization (He)	–	–	–	–
Best Regularization	8.33	–	–	–
Best Optimizer (RMSProp)	–	–	–	–
Best Hyperparameters (C2)	13.00	9.27	4.36	5.32
Fine-Tuned Model	–	–	4.36	5.32

Overall Inference:

Among the cross-validation configurations, C2 achieved the highest mean validation accuracy of 13.00%, although it also had a relatively high standard deviation of 9.27%. The final model achieved a test accuracy of 4.36%. The low overall accuracy is mainly due to the very small dataset subset and the use of only one training epoch in the fast, memory-optimized implementation.

The RMSProp optimizer achieved the highest validation accuracy among the optimizers, with 11.67% validation accuracy and a final loss of 1.4825. For regularization, the model without regularization achieved the highest validation accuracy of 8.33%.

The final model used learning rate 10^{-4} , dropout 0.25 and batch size 32, corresponding to configuration C2. It achieved a test accuracy of 4.36%, macro precision of 0.0220, macro recall of 0.0435 and macro F1-score of 0.0182.

The total number of parameters was 2,271,269, of which 47,397 were trainable.

14. Required Inference for Plots

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?

Each inference should be approximately 2–3 lines.

15. Discussion Questions

1. What is the difference between model parameters and hyperparameters?
2. Why is weight initialization important?
3. Why can zero initialization be problematic for neural networks?
4. Compare Xavier and He initialization.
5. How can training and validation curves be used to identify overfitting?
6. How does Dropout reduce overfitting?
7. What is the purpose of Batch Normalization?
8. Explain the numerical Batch Normalization example.
9. What are the roles of γ and β ?
10. Compare SGD, Momentum, RMSProp and Adam.
11. What happens when the learning rate is too large?
12. What happens when the learning rate is too small?
13. What is the effect of increasing batch size?
14. Explain stride and padding.
15. Why is MobileNetV2 computationally efficient?
16. What is depthwise separable convolution?
17. What is transfer learning?
18. Differentiate feature extraction and fine-tuning.
19. Why is a smaller learning rate generally used during fine-tuning?
20. Why is K-Fold Cross-Validation useful for hyperparameter selection?

21. Why must the test set remain untouched during tuning?
22. Why should mean and standard deviation both be reported?
23. Is the highest validation accuracy always sufficient to select a model?

16. Additional Exercise

Select two new combinations of learning rate, dropout, batch size and fine-tuning strategy. Evaluate them using 5-fold cross-validation and compare their results with the selected configuration.

Students must justify whether the new configuration is preferable based on accuracy, standard deviation, computational cost and final test performance.

17. Expected Outcome

Students should experimentally demonstrate that CNN performance is strongly influenced by initialization, regularization, optimization and hyperparameter choices. They should understand the MobileNetV2 architecture, perform transfer learning and fine-tuning, and select a reliable final configuration using 5-fold cross-validation.

18. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, “Cats and Dogs,” IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>